

Analizador Léxico em Python

Arthur Emanuel da Silva^{1*}, Jéssica Brito da Silva^{1*}

¹Curso de Ciência da Computação – Universidade Federal da Fronteira Sul (UFFS)
Chapecó, SC – Brazil

{arthurwyxzfswi, jessicaburitto}@gmail.com

Resumo. *O presente trabalho visa a construção de um Analizador Léxico utilizando a linguagem Python que, a partir de uma entrada contendo strings arbitrárias, um conjunto de tokens e uma gramática regular, construa o Automato Finito Não Determinístico (AFND), faça sua determinização e realize o mapeamento do Autômato Finito Determinístico (AFD) resultante em um reconhecedor léxico. O algoritmo visa, dada as entradas, retornar uma fita de saída de estados e uma Tabela de Símbolos contendo: linha, identificador e rótulo*

Abstract. *The present work aims to build a Lexical Analyzer using the Python language which, from an input containing arbitrary strings, a set of tokens, and a regular grammar, constructs the Non-Deterministic Finite Automaton (NFA), performs its determinization, and maps the resulting Deterministic Finite Automaton (DFA) into a lexical recognizer. The algorithm aims, given the inputs, to return an output state tape and a Symbol Table containing: line, identifier, and label.*

1. Introdução

O objetivo central deste trabalho é o desenvolvimento de um analisador léxico, que atua como a primeira etapa de um compilador, sendo responsável por ler um código-fonte e agrupar os caracteres em unidades lógicas chamadas tokens. Basicamente, o analisador verifica se as palavras digitadas pertencem à linguagem definida, separando o que são palavras reservadas de identificadores válidos. Nesse projeto, isso é feito de forma automatizada: o algoritmo lê um arquivo de tokens e uma gramática regular para construir um Autômato Finito Não-Determinístico (AFND). Em seguida, esse autômato passa por um processo de determinização para se transformar em um AFD, o que realmente permite ao programa decidir, sem ambiguidades, qual caminho seguir para cada caractere lido. Ao final do processo, o sistema gera uma fita com os estados percorridos e uma Tabela de Símbolos que organiza informações como a linha, o rótulo do token e se ele foi aceito ou se resultou em um estado de erro.

2. Fundamentação Teórica

O desenvolvimento de um analisador léxico fundamenta-se na Teoria da Computação, especificamente no estudo das Linguagens Formais e dos Autômatos. Segundo [Scheffel 2005], um compilador é dividido em fases, sendo a análise léxica a primeira delas. Essa fase se baseia em três pilares fundamentais:

- **Alfabetos e Linguagens:** Alfabetos são conjuntos finitos de símbolos. A combinação desses símbolos forma cadeias (strings). No contexto deste trabalho,

o alfabeto é restrito aos caracteres que compõem os tokens da nossa linguagem específica, como as letras de "se", "sai", "foi" e os caracteres definidos para as variáveis.

- **Gramáticas Regulares (GR):** Para definir quais palavras são válidas em uma linguagem, utilizamos as Gramáticas. No caso dos analisadores léxicos, trabalhamos com as Gramáticas Regulares, que são o nível mais simples da Hierarquia de Chomsky. Elas utilizam regras de produção para dizer como uma palavra deve começar e como ela pode continuar. Por exemplo, a regra $S \rightarrow aA$ indica que, se lermos um 'a', passamos para um novo estado de espera 'A'.
- **Autômatos Finitos (AF):** O autômato é o mecanismo prático que "executa" a gramática. Ele é um modelo matemático composto por estados e transições. Em um AFND (Não-Determinístico) à partir de um estado, o computador poderia teoricamente seguir vários caminhos para o mesmo símbolo o que torna mais difícil de programar diretamente. Já no AFD (Determinístico) para cada símbolo lido em um estado, existe exatamente um único caminho possível. O processo de determinização (transformar AFND em AFD) é crucial para que o algoritmo de reconhecimento seja eficiente e sem ambiguidades.

O reconhecimento léxico ocorre através do mapeamento do AFD em código. O programa consome a entrada caractere por caractere, saltando entre os estados conforme a tabela de transição. Se, ao encontrar um separador (como um espaço), o programa estiver em um estado de aceitação, o token é reconhecido e enviado para a Tabela de Símbolos. Caso contrário, ou se ele cair em um estado de erro, o sistema sinaliza que a cadeia é inválida para aquela linguagem.

3. Resultados e Conclusões

4. Implementação e Resultados

Nesta seção, descreve-se o processo de desenvolvimento do reconhecedor léxico, que foi dividido em três etapas principais: especificação da linguagem, construção do motor de estados e execução da análise.

4.1. Especificação de Tokens e Gramática

A base do projeto utiliza um arquivo de configuração externo (`tokens.txt`). Nele, foram definidas palavras reservadas como "se", "sai" e "foi", além de uma Gramática Regular (GR) para identificadores. A regra gramatical estabelece que identificadores devem iniciar com caracteres específicos (f, a, e, i) e podem ser seguidos por sequências dos mesmos caracteres ou encerrar-se.

4.2. Construção do Autômato (`afd.py`)

A primeira parte lógica do sistema lê essas definições e constrói um Autômato Finito Não-Determinístico (AFND). Como o reconhecimento computacional exige precisão, o algoritmo realiza a determinização, transformando o AFND em um Autômato Finito Determinístico (AFD).

- **Decisão de Projeto:** O sistema utiliza um dicionário de estados em Python para representar a matriz de transição, onde cada par (Estado Atual, Caractere) leva a exatamente um novo estado.

- **Tratamento de Erros:** Foi implementado um estado de erro (representado por ~ ou X) para onde o fluxo é desviado caso um caractere inválido ou uma transição não permitida ocorra.

4.3. Algoritmo de Reconhecimento Léxico (a1.py)

Com o AFD pronto, o analisador processa o arquivo de entrada (`entrada.txt`) seguindo o algoritmo de mapeamento:

1. **Leitura:** O texto é lido palavra por palavra, utilizando espaços e quebras de linha como separadores.
2. **Transição:** Para cada caractere, o programa consulta a tabela de estados.
3. **Validação:** Ao final de cada palavra, verifica-se se o estado atual é um estado final de aceitação.

4.4. Resultados e Validação

O sistema gera duas saídas principais exigidas: a Fita de Saída e a Tabela de Símbolos. A Fita exibe a sequência de identificadores de estados para cada token reconhecido. A Tabela de Símbolos organiza o rótulo original, a linha correspondente e o status (aceito ou erro).

```
ALFABETO: ['a', 'b', 'e', 'f', 'i', 'o', 'r', 's', 't']
MAPEAMENTO: {'a': 0, 'b': 1, 'e': 2, 'f': 3, 'i': 4, 'o': 5, 'r': 6, 's': 7, 't': 8}
ESTADO FINAL: {'9,13', '8', '5', '2', '12', '13'}
```



```
--- TABELA DE SÍMBOLOS ---
```

ENTRADA	LINHA	RÓTULO	SEQUENCIA DE ESTADOS
se	0	aceito	0 -> 1,3 -> *2
sai	0	aceito	0 -> 1,3 -> 4 -> *5
foi	0	aceito	0 -> 6 -> 7 -> *8
as	1	ERRO	0 -> ~ -> ~
raie	1	aceito	0 -> *13 -> *13 -> *13 -> *13
bia	1	aceito	0 -> 9,*13 -> *13 -> *13
biii	2	aceito	0 -> 9,*13 -> *13 -> *13 -> *13
jessica	2	ERRO	0 -> ~ -> ~ -> ~ -> ~ -> ~ -> ~ -> ~
jesca	3	ERRO	0 -> ~ -> ~ -> ~ -> ~ -> ~
jes	3	ERRO	0 -> ~ -> ~ -> ~
brie	3	aceito	0 -> 9,13 -> 10 -> 11 -> *12


```
--- FITA DE SAÍDA ---
```

```
2 5 8 ~ 13 13 13 ~ ~ ~ 12 0
```

Figura 1. Fita de Estados e a Tabela de Símbolos

[To]

Os testes demonstraram que o analisador identifica corretamente as palavras reservadas e variáveis que seguem a GR, enquanto desvia cadeias inválidas para o estado de erro, cumprindo integralmente os requisitos do projeto.

```
≡ entrada.txt
1   se sai foi
2   as raie bia
3   biii jessica
4   jesca jes brie
```

Figura 2. Entrada

```
≡ tokens.txt
1   se
2   sai
3   foi
4   brie
5
6   <S> ::= b<A> | r<A> | t<A>
7   <A> ::= e<A> | i<A> | a<A> | *
```

Figura 3. Tokens e Gramática Regular

5. References

Referências

Scheffel, R. M. (2005). Linguagens formais e autômatos. Apostila didática.